

Основи ООП. Поняття класів та об'єктів. Робота з класами.

1. Що таке ООП?

Об'єктно-орієнтоване програмування (ООП) — це спосіб створення програм, де основними елементами є **об'єкти**.

Програма → набір взаємодіючих об'єктів.

2. Основні поняття ООП

2.1. Клас

Клас — це **шаблон**, опис майбутніх об'єктів.

Містить:

дані (поля / атрибути)
функції-члени (методи)

2.2. Об'єкт

Об'єкт — це **екземпляр класу**.

Створюється за шаблоном класу.

3. Принципи ООП

Інкапсуляція — приховування реалізації, відкриття лише потрібних частин.

Наслідування — створення нового класу на основі існуючого.

Поліморфізм — можливість виклику різних реалізацій через один інтерфейс.

(Подробиці будуть в наступних лекціях.)

4. Створення класу в C++

```
class MyClass {  
private:  
    int a;    // приватні дані
```

```
public:
    void setA(int x) { a = x; } // метод
    int getA() { return a; }
};
```

5. Об'єкти класу

```
MyClass obj;
obj.setA(10);
cout << obj.getA();
```

6. Модифікатори доступу

Ключове слово	Доступ
private	тільки всередині класу
public	доступний усім
protected	клас + нащадки

7. Конструктори

Конструктор — це спеціальний метод, який викликається автоматично при створенні об'єкта.

```
class A {
public:
    A() { cout << "Constructor"; }
};
```

Можуть бути:

- без параметрів
- з параметрами
- копіювання
- з ініціалізацією полів:

```
A(int x, int y) : a(x), b(y) {}
```

8. Деструктор

Викликається при знищенні об'єкта.

```
~A() {  
    cout << "Destructor";  
}
```

Важливий при роботі з ресурсами (пам'ять, файли).

9. Ключове слово **this**

this — це вказівник на поточний об'єкт.

```
void setX(int x) {  
    this->x = x;  
}
```

Використовується для:

- уникнення конфлікту імен
- повернення самого об'єкта (return *this;)

10. Розділення оголошення та реалізації класу

У заголовочному файлі .h:

```
class MyClass {  
public:  
    void f();  
};
```

У cpp-файлі:

```
void MyClass::f() {  
    cout << "Hi!";  
}
```

11. Створення об'єктів у стеку й у heap

11.1. На стеку

A obj;

Автоматичне створення та знищення.

11.2. У heap

```
A* p = new A();  
delete p;
```

Керуємо життєвим циклом вручну.

12. Передача об'єктів у функції

За значенням — копія:

```
void f(A a);
```

За посиланням:

```
void f(A& a);
```

За вказівником:

```
void f(A* a);
```

13. Об'єкти як значення, а не тільки як вказівники

На відміну від Java/C#, у C++ об'єкти **можуть жити на стеку**, не лише у heap.

Це ключова особливість мови.

14. Перевантаження методів

Методи з однаковим ім'ям, але різними параметрами:

```
void set(int a);  
void set(double a);  
void set(int a, int b);
```

C++ обирає потрібну версію автоматично.

15. Приховання полів

Коректний спосіб взаємодії з полями — через **геттери та сеттери**:

```
int getX();  
void setX(int x);
```

16. Статичні поля та методи

Статичне поле (спільне для всіх об'єктів):

```
static int count;
```

Статичний метод:

```
static void foo();
```

Виклик:

```
A::foo();
```

17. Пам'ять об'єктів

Об'єкт складається з:

полів (даних)

таблиці віртуальних функцій (якщо є virtual)
вирівнювання пам'яті (alignment)

18. Що таке ОО-дизайн?

Створення системи у вигляді взаємодіючих класів та об'єктів.

Принципи:

- абстракція
- інкапсуляція
- зв'язність і слабка залежність
- повторне використання коду
- модульність